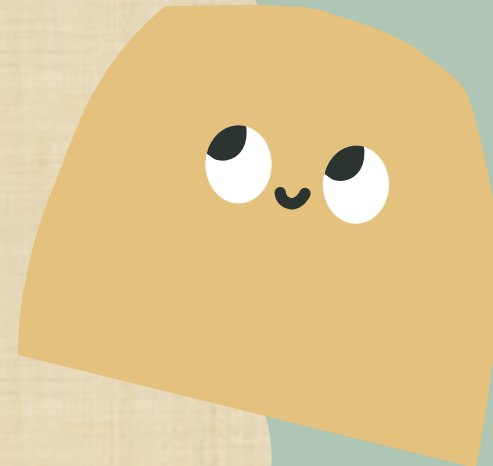
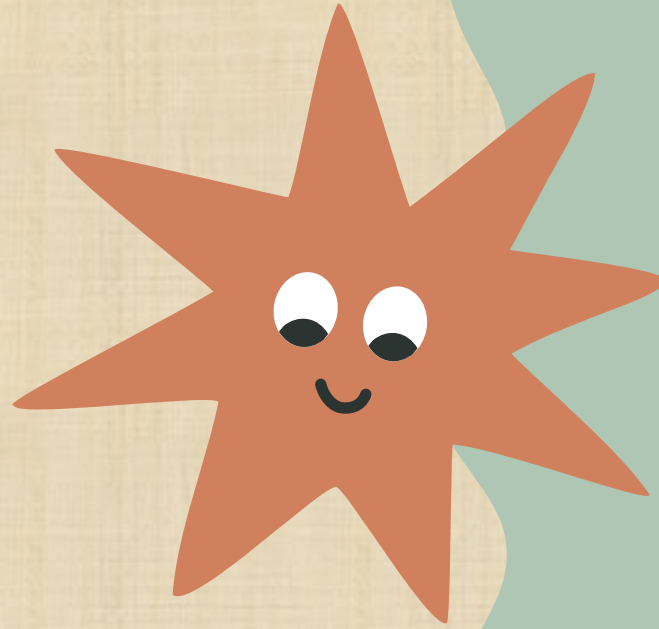


UNIT 4 CONTROL STRUCTURE

10TH COMPUTER SCIENCE
SINDH TEXTBOOK BOARD

<https://awinashgoswami.github.io>



Agenda

- Control Statement
- Selection/Decision Making Structure
- Iteration/Loop
- Jump Statement

CONTROL STATEMENTS

- A computer program is the set of instructions in sequential form. These instructions execute from top to bottom. We can control the flow of program with the help of Control Statements. Control Statements are used to control the direction of program by repeating any block of code, making a choice or simply transfer the control to any specific block of program. These statements help programmers to decide which part of code is executed at certain time.
- C++ has three types of control statements:
 - Selection/Decision Making Structure
 - Iteration / Loops
 - Jump

SELECTION/DECISION MAKING STRUCTURE

- They are used to decide whether a certain part of code is executed or not.
- C++ has three decision making structures:
 - 'if' statement
 - 'if-else' statement
 - 'switch' statement

"if" statement

- It is the basic decision statement. The structure contains "if" keyword followed by a conditional expression in parenthesis and then its body of statements(s) also called if block. If there are more than one statement in "if" block or body we enclose all of them in braces.
- If statement checks a condition, if the condition is true the statements in "if" block are executed and if condition is false it leaves the statements in "if" block and starts executing statements after "if" block.

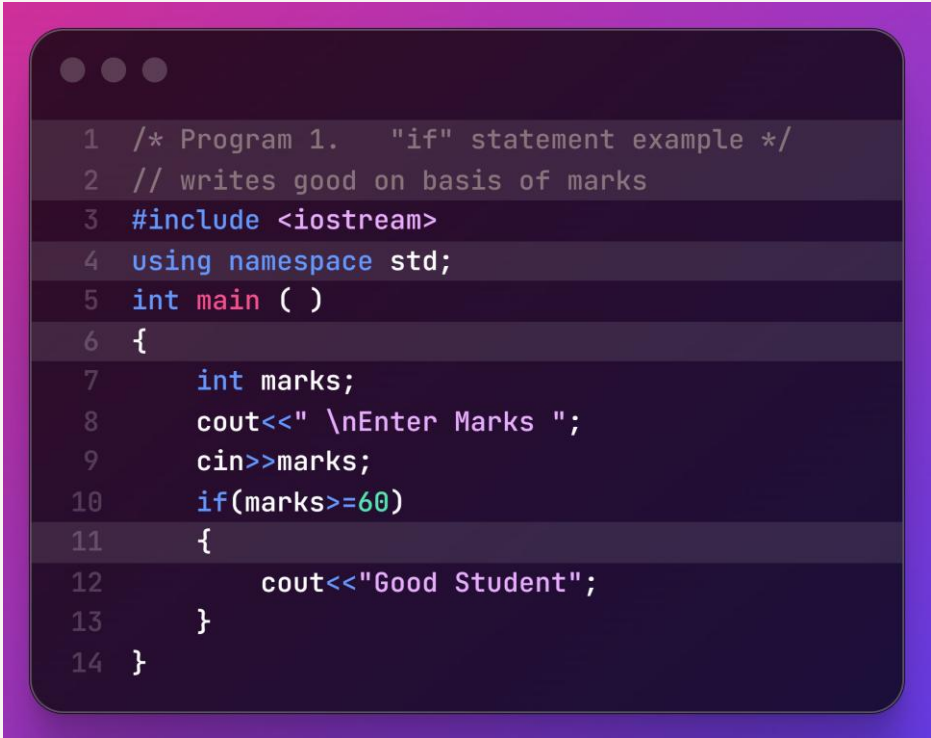
Syntax

```
if (test condition)

{

    Statement(s) ;

}
```



```
1  /* Program 1.   "if" statement example */
2  // writes good on basis of marks
3  #include <iostream>
4  using namespace std;
5  int main ( )
6  {
7      int marks;
8      cout<<" \nEnter Marks ";
9      cin>>marks;
10     if(marks>=60)
11     {
12         cout<<"Good Student";
13     }
14 }
```

Nested "if" Statement

- An "if" statement which is part of block of another 'if' statement is called nested 'if'. The inner 'if' statement will only be tested if the outer 'if' is true.
- In following program, marks and age of a person are taken as input. First condition checks marks, if they are greater than or equal to 60 then next condition checks age. If age is also greater than 18 then it prints message "you got the job". "good luck" is always printed.

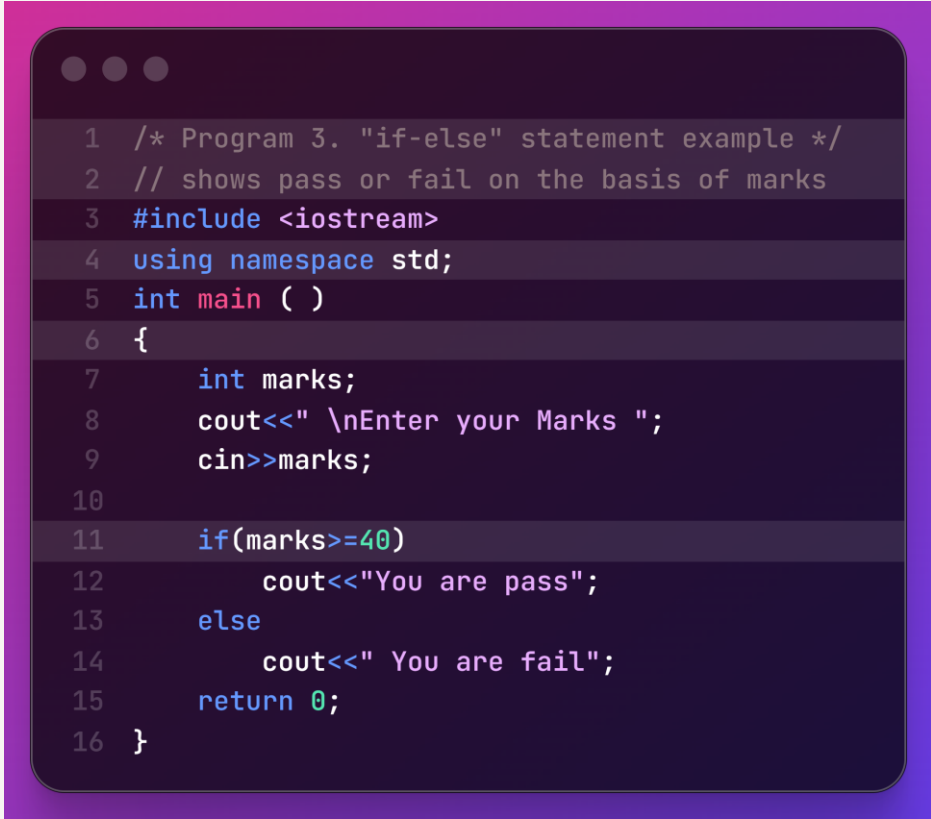
```
/* Program 1.  "if" statement example */  
// writes good on basis of marks  
#include <iostream>  
using namespace std;  
int main ( )  
{  
    int marks;  
    cout<<" \nEnter Marks ";  
    cin>>marks;  
  
    cout<<" You are a ";  
    if(marks>=60)  
    {  
        cout<<" good ";  
    }  
    cout<<"Student of this class";  
    return 0;  
}
```

"if-else" statement

- The structure of if- else contains 'if' keyword followed by a conditional expression in parenthesis and then its body of statements(s) then 'else' keyword and its body of statements. It checks the condition, if the condition is true the statements in if block are executed and in case of false condition, it executes statements in 'else' block. It means either 'if' block statements or 'else' block statements must execute.

Syntax

```
if (test condition)
{
    Statement(s);
}
else
{
    Statement(s);
}
```



```
1  /* Program 3. "if-else" statement example */
2  // shows pass or fail on the basis of marks
3  #include <iostream>
4  using namespace std;
5  int main ( )
6  {
7      int marks;
8      cout<<" \nEnter your Marks ";
9      cin>>marks;
10
11     if(marks>=40)
12         cout<<"You are pass";
13     else
14         cout<<" You are fail";
15     return 0;
16 }
```

else-if statement

- In "if" statement if we use nesting deeply i.e. one if is nested to other and other is nested in another and so on then in indentation, we see a ladder like structure which is difficult to understand like below. Here is an example. This program takes marks as input and then determines the grade by applying if condition multiple times.

```
/* Program 4. "else-if" statement example */
// shows Grade on the basis of marks
#include <iostream>
using namespace std;
int main ( )
{
    int marks;

    cout<<" \nEnter your Marks ";
    cin>>marks;

    if(marks>=80)
        cout<<"Grade is A1";
    else if(marks>=70)
        cout<<" Grade is A";
    else if(marks>=60)
        cout<<" Grade is B";
    else if(marks>=50)
        cout<<" Grade is C";
    else
        cout<<" Fail";
    return 0;
}
```

"switch" statement

Syntax

```
switch(expression)
{
    case constant 1:
        statement(s)
        break;
    case constant 2:
        statement(s)
        break;
    default:
        statement(s)
}
```

```
1  /* Program 5.  "switch" statement example */
2  // prints name of day checking day number
3  #include <iostream>
4  using namespace std;
5  int main ( )
6  {
7
8      int day;
9
10     cout<<"\nEnter number of weekday 1 to 7 ";
11     cin>>day;
12
13     switch(day)
14     {
15         case 1: cout<<"Sunday"; break;
16         case 2: cout<<"Monday"; break;
17         case 3: cout<<"Tuesday"; break;
18         case 4: cout<<"Wednesday"; break;
19         case 5: cout<<"Thursday"; break;
20         case 6: cout<<"Friday"; break;
21         case 7: cout<<"Saturday"; break;
22         default: cout<<"Invalid day number";
23     }
24
25     return 0;
26 }
```

Difference between If, If-Else and switch

"if"	"if" statement checks a condition using relational and other operators, if the condition is true the statements in if block execute. If condition is false, it leaves the statements in "if" block and starts executing statements after "if" block.
"if-else"	"if-else" statement checks a condition using relational and other operators, if the condition is true the statements in "if" block are executed and in case of false condition it executes statements in 'else' block. It means either "if" block statements will execute or "else" block statements.
"switch"	"switch" statement checks 'switch' variable value equal to constant that follows case statement. If it matches to any of them then control is transferred to that case and all statement after colon are executed and switch is broken with "break" statement. Otherwise, control is transferred to "default" statement. It has some limitations. It only matches character and integer data type variables, it checks switch variable value only with case constants not with any variable and it also cannot use relational operators like less than (<) or greater than (>) and exactly matches value with case constants.

ITERATION/LOOP

- Normally statements are executed sequentially, one after the other. In some situations, we need to execute a block of statements several number of times. Loops allow us to execute a statement or a group of statements several number of times. A group or block is made by enclosing statements in braces. There are three types of loops in C++.
 - **for**
 - **while**
 - **do- while**

"for" loop

- For loop is usually used in situations where at the start of loop we know that how many times loop block will execute. It is a pre-test loop as condition is tested at the start of loop.
- First part is initialization which initializes loop variable; second part is test expression which tests loop variable and third part is increment or decrement in loop variable. Then comes the body of the loop which may have one or more statements. If there are more than one statements then they are enclosed in braces.

Syntax

```
for (initialization; condition testing; increment/decrement)
{
    statement(s);
}
```



```
1  #include<iostream>
2  using namespace std;
3  int main()
4  {
5      for(int num=1; num<=10; num++){
6          cout<<"Number: "<<num<<endl;
7      }
8  }
```

"while" loop

- It is a pre-test loop. It tests the condition at the start of loop before executing the body of loop. It is usually used in situations where we do not know at the start of loop, how many times loop block will execute. So, it is also called indefinite repetition loop. It starts with keyword **"while"** followed by a conditional expression like if statement in parenthesis. Then comes the body of the loop which may have one or more statements in braces.

Syntax

```
while (condition)

{

    statement (s) ;

}
```



```
1  #include<iostream>
2  using namespace std;
3  int main()
4  {
5      int num = 1;
6      while(num<=10){
7          cout<<"Number: "<<num<<endl;
8          num++;
9      }
10 }
```

"do while" loop

- It is similar to while loop, except that it tests the condition at the end of the loop body and so it is also called post-test loop. Its statements block is executed at least once. For second time condition is tested. It starts with keyword **"do"** follow by a body of loop which may have one or more statements in braces. Then **"while"** keyword and conditional expression in parenthesis. It must be terminated with semi colon.

Syntax

do

{

Statement 1;

Statement 2;

Statement 3;

}

while(condition);

```
1  #include<iostream>
2  using namespace std;
3  int main()
4  {
5      int num = 1;
6      do {
7          cout<<"Number: "<<num<<endl;
8          num++;
9      }
10     while(num<=10);
11 }
```

Nested loops

- You can use one or more loop inside any another 'for', 'while' or 'do while' loop. If a loop exists in the body of another loop, then it is called nested loop. The inner nested loop is completely executed every time for each repetition of outer loop.
- The following program is of nested for loop. It prints numbers from 1 to 10 in five rows. The outer loop shows row number, then inner loop prints numbers from 1 to 10 in one row then outer loop changes the row using '\n'. The outer loop runs five times, so inner loop which prints 1 to 10 numbers will run five times and prints numbers in five rows.

```
1  /* Program 9.  nested loops example */
2  // prints numbers from 1 to 10 in five rows
3  #include <iostream>
4  using namespace std;
5  int main ( )
6  {
7
8      int i,j;
9
10     for(i=1;i<=5;i++)
11     {
12         cout<<" Row no. "<<i<<" -> ";
13         for(j=1;j<=10;j++)
14             cout<<j<<" ";
15         cout<<"\n";
16     }
17     return 0;
18 }
```

Difference between for, while and do-while loop structures

"for" Loop	"while" Loop	"do while" Loop
"for" loop is usually used in situations where we know at the start of loop that how many times loop block will execute. It is also called definite repetition loop.	It is usually used in situations where we do not know at the start of loop that how many times loop block will execute. It is also called indefinite repetition loop.	It is usually used in situations where we do not know at the start of loop that how many times loop block will execute. It is also called indefinite repetition loop.
It is called counter controlled loop as loop is controlled by a counter value, at each iteration counter value will increase or decrease	It does not need a counter value for its execution.	It does not need a counter value for its execution.
It has three parts first initialization, second condition testing and third increment or decrement.	It has only one part which is condition testing. If needed initialization is done before loop and increment or decrement is done in loop body.	It has only one part which is condition testing. If needed initialization is done before loop and increment or decrement is done in loop body.
It is pre-test loop as condition is tested at start of loop.	It is pre-test loop as condition is tested at start of loop.	It is post-test loop as condition is tested at the end of loop. So this loop executes at least once

JUMP STATEMENTS

- Jump statements change execution of program from its normal sequence.
- Following are the jump statements used in C++
 - break
 - continue
 - goto
 - return
 - exit ()

"break" Statement

- It terminates the loop or switch statement and transfers control to the statement immediately following the loop or switch statement.

```
1  #include<iostream>
2  using namespace std;
3  int main()
4  {
5      for(int i=1; i<=10; i++){
6          cout<<"Number: " <<i <<endl;
7          if(i==6)
8              break;
9      }
10 }
```

"continue" Statement

- It causes the loop to skip the remaining statements of its body and immediately transfers control to the top of the loop i.e. first statement.

```
1  /* Program 11.   "continue" statement example */
2  // Adds five positive numbers. It does not take -ve numbers
3  #include <iostream>
4  using namespace std;
5  int main ( )
6  {
7      int i, num, sum;
8      i=0; sum=0;
9      cout<<"\n Enter five positive numbers-> ";
10     while(i<5)
11     {
12         cin>>num;
13         if(num<=0)
14         {
15             cout<<"\n Enter positive number";
16             continue;
17         }
18         sum=sum+num;
19         i++;
20     }
21     cout<<"\n sum of five positive numbers is "<<sum;
22     return 0;
23 }
```

"goto" Statement

- A "goto" statement jumps or transfers control unconditionally from the "goto" to a labeled statement in the same function. A labeled statement is any identifier followed by a colon (:). It is not advised to use "goto" statements in programs.

```
#include <iostream>
using namespace std;

int main() {
    int age;
    cout << "Enter your age: ";
    cin >> age;

    if (age < 0)
        goto warning_negative;

    if (age > 150)
        goto warning_too_high;

    cout << "Age accepted: " << age << "\n";
    goto end;

warning_negative:
    cout << "⚠ Warning: Age cannot be negative!\n";
    goto end;

warning_too_high:
    cout << "⚠ Warning: Age seems unrealistic (> 150)!\n";
    goto end;

end:
    cout << "Program finished.\n";
    return 0;
}
```

"return" statement

- It terminates the execution of a function and transfers program control to the statement just after the function call statement in the calling function (or to the operating system if you transfer control from the main function). It can also return a value from the current function, if return type is not "void".
- **Syntax:** return [expression/value];
- The value of the expression clause is returned to the calling function.

```
#include <iostream>
using namespace std;

// Function that returns early with a warning
int divide(int a, int b) {
    if (b == 0) {
        cout << "⚠ Warning: Division by zero!\n";
        return -1;           // jump out of the function immediately
    }
    return a / b;           // normal return
}

int main() {
    cout << "Result: " << divide(10, 2) << "\n";
    cout << "Result: " << divide(10, 0) << "\n";
    cout << "Back in main.\n";
    return 0;
}
```

"exit ()" function

- The exit() is used to terminate a C++ program and closes program whenever executed. It is defined under "stdlib.h" header file.

```
#include <iostream>
#include <cstdlib> // needed for exit()
using namespace std;

int main() {
    int a = 10, b = 0;

    if (b == 0) {
        cout << "⚠ Fatal error: Division by zero!\n";
        exit(1); // terminates the whole program right here
    }

    cout << "Result: " << a / b << "\n";
    cout << "This line will NEVER run.\n";

    return 0;
}
```